

Python for Web Development

Lesson 1: Setting Up the Development Environment

Setting Up the Development Environment - Study Notes

Key Concepts

1. **Python installation & version management** – choosing the right Python release, verifying the install, and handling multiple versions.
2. **Virtual environments (virtualenv/venv)** – isolating project dependencies, creating, activating, and deactivating environments.
3. **Integrated Development Environment (IDE) selection & configuration** – popular IDEs/editors, essential plugins/extensions, and workspace setup.
4. **Project skeleton structure** – standard directory layout, placement of source code, tests, documentation, and configuration files.
5. **Dependency management** – using `requirements.txt` or `pyproject.toml` to lock package versions for reproducibility.

Summary

Setting up a reliable development environment is the foundation for any Python project. Begin by downloading the latest stable Python release from python.org (or using a version manager like `pyenv`) and confirming the installation with `python --version` and `pip --version`. Once Python is ready, create an isolated virtual environment for each project using `python -m venv <env_name>` (or the third party `virtualenv` for older Python versions). Activating the environment ensures that all `pip` installs affect only that project, preventing version clashes across different code bases.

Choosing an IDE or editor that matches your workflow speeds up development. VS /Code, PyCharm, and Sublime Text are common choices; each offers syntax highlighting, linting, debugging, and integrated terminal support. Install language specific extensions (e.g., Python extension for VS /Code) and configure the interpreter to point at the virtual environment you created. Finally, scaffold a clean project skeleton—typically a top level package directory, a `tests/` folder, a `README.md`, a `requirements.txt` (or `pyproject.toml`), and a `.gitignore`. This structure promotes consistency, makes onboarding new contributors easier, and aligns with best practices for packaging and distribution.

Important Points

- **Download Python** from the official site or use a version manager; avoid system wide installs on Linux/macOS when possible.
- **Add Python to PATH** (Windows) or ensure `python3` resolves correctly (Unix).
- **Create a virtual environment**:

```
``bash
python -m venv .venv      # creates .venv directory
source .venv/bin/activate # macOS/Linux
```

```
.venv\Scripts\activate    # Windows
```

```
---
```

- **Deactivate** with `deactivate``.
- **Never commit the virtual env folder**; add it to `.gitignore``.
- **IDE configuration**: set the interpreter to the virtual env's Python binary; enable linting (flake8, pylint) and auto formatting (black).

- **Project skeleton** (example):

```
---
```

```
my_project/
% % .venv/          # (ignored)
% % src/
% % % my_package/
% % % __init__.py
% % % core.py
% % tests/
% % % test_core.py
% % .gitignore
% % README.md
% % requirements.txt
% % pyproject.toml # optional, for Poetry/PEP 517 builds
```

```
---
```

- **Pin dependencies** in `requirements.txt`` (`pip freeze > requirements.txt``) to guarantee reproducible installs.
- Use **source control** (Git) from the start; commit the skeleton and lock files, not the environment.

```
---
```

Examples

Example 1 – Installing Python & Creating a Virtual Environment

```
```bash
```

## 1. Download and install Python 3.11 (Windows installer)

## 2. Verify installation

```
python --version # => Python 3.11.x
```

```
pip --version # => pip 23.x
```

## 3. Create project folder

```
mkdir my_app && cd my_app
```

## 4. Set up virtual environment

```
python -m venv .venv
```

## 5. Activate it

### macOS/Linux

```
source .venv/bin/activate
```

### Windows

```
.\venv\Scripts\activate
```

## 6. Install a package inside the env

```
pip install requests
```

## 7. Freeze dependencies

```
pip freeze > requirements.txt
```

```

```

\*Explanation:\* Steps 1-5 give you an isolated Python interpreter. Step 6 shows that any `pip install` now affects only this project. Step 7 records exact versions for later reproducibility.

### Example 2 – VS /Code Configuration for the New Project

1. Open the `my\_app` folder in VS /Code (`code .`).
2. Install the **Python** extension (Microsoft).
3. Press `Ctrl+Shift+P` ! **Python: Select Interpreter** ! choose `.venv/bin/python` (or `.venv\Scripts\python.exe`).
4. Create a `.vscode/settings.json` file to enforce formatting and linting:

```
```json
{
  "python.formatting.provider": "black",
  "python.linting.enabled": true,
  "python.linting.pylintEnabled": true,
  "python.testing.unittestEnabled": true,
  "python.testing.pytestEnabled": false
}
```
```

5. Add a basic test file in `tests/test\_core.py` and run it via the **Testing** sidebar.

\*Explanation:\* VS /Code now uses the project's virtual environment, applies Black for auto formatting, and runs pylint on save. The test explorer can discover and run the unit tests you add.

```

```

## Quick Review

1. Why should you use a virtual environment for each Python project?
2. What command creates a virtual environment using the built-in `venv` module?
3. Name two popular IDEs/editors for Python and one key extension each provides.

4. List three top level items you would include in a fresh project skeleton.
5. How do you record the exact versions of installed packages for future reproducibility?

---

## Further Reading

- **Python Packaging Guide** – <https://packaging.python.org/tutorials/packaging-projects/>
- **PEP /508 & Dependency Specification** – <https://peps.python.org/pep-0508/>
- **pyenv & pyenv virtualenv** – managing multiple Python versions on Unix like systems.
- **Poetry** – modern dependency management and packaging tool (alternative to `requirements.txt`).
- **Testing frameworks** – `unittest`, `pytest`, and test discovery in IDEs.

